Cross-Device Translation Interference in a RISC-V IOMMU: A Cycle-Level Characterization

Author information pending
[TODO: 需要作者决定: authors, affiliations, contact email]

ABSTRACT

A shared input/output memory management unit (IOMMU) can expose one device's address translations to another device's invalidations and replacement activity. We characterize these effects using existing cycle-level simulations of direct memory access (DMA) engines and a RISC-V IOMMU, with a memory command queue and passive translation tracing. In the tested synthetic copies, broad invalidation increases a persistently mapped neighbor's completion time by 14.8%–17.6%, including across distinct first-stage address spaces. Context-scoped invalidation across domains, or context-and-address-scoped invalidation within a domain, eliminates invalidation-attributed misses, while increasing the initiator's completion time by 9.7%–10.0% relative to broad invalidation. Batching unmaps provides partial relief because map-side invalidations remain. A separate ablation shows that our platform's way-masked tree-PLRU extension underuses its nominal partition capacity; changing its replacement policy reduces completion time by 12.44%–22.37% in the affected case. This masked-tree problem does not apply to the upstream unpartitioned implementation. Mainline Linux already supports address-scoped and deferred invalidation; this study quantifies policy choices rather than identifying a mainline defect. In a separate mainline-derived model, strict unmap increases neighbor time by 2.24%–2.47%; deferred flushing and ideal dead-entry removal match the no-invalidation baseline within the short working window. The results use behavioral memory timing, with neither CPU nor Linux execution, and do not establish hardware or application performance.

KEYWORDS

RISC-V, IOMMU, DMA, translation cache, invalidation, RTL simulation

1 INTRODUCTION

An input/output memory management unit (IOMMU) translates device-visible input/output virtual addresses (IOVAs), using page tables and cached translations [7]. When direct memory access (DMA) engines share this structure, a neighbor can lose translations through replacement or through an invalidation whose scope includes its address space. These mechanisms call for different remedies: reserving cache entries addresses replacement eligibility, whereas invalidation scope determines which resident entries software removes. Prior work studies IOMMU translation costs, sequential ring-buffer translation, translation prefetching, and translation-aware memory scheduling [1, 2, 4, 6]. Our question is concrete: in one open RISC-V implementation, which shared-translation paths affect a neighbor, by how much under a fixed workload, and with what completion cost to the initiator?

We use the DMA and IOMMU components of a CVA6-based platform, with real register accesses and command-queue fetches in register-transfer-level (RTL) simulation. The main experiment preserves the upstream translation-cache and command-handler logic while holding documented platform adaptations and functional fixes constant. It compares a no-invalidation baseline against broad, PSCID-scoped, address-scoped, and delayed-unmap invalidation. Passive observation records actual resident-entry removal and the first subsequent miss, allowing a distinction between invalidation-attributed misses and other misses.

The principal result is conditional: broad invalidation dominates the neighbor's translation interference in this workload, even with separate host domains. Narrowing its scope protects the neighbor but increases the initiating device's complete time in these runs. We also report a separate partition-extension ablation as a design caution: a way mask on a tree replacement policy can provide less effective residency than its nominal quota suggests. These are characterization results; they establish neither a new replacement algorithm nor performance across IOMMU implementations.

The Linux distinction is central to the claim. At the audited mainline revision, ordinary invalidation already uses PSCID and address scope, with domain-scoped fallback and deferred-flush support; broad host invalidation is the behavior of the audited development branch and local derivative. The paper quantifies the consequences of those choices rather than presenting broad invalidation as a defect in current mainline Linux.

2 BACKGROUND

2.1 Translation and invalidation scope

The RISC-V IOMMU specification defines device contexts, first-stage process soft-context identifiers (PSCIDs), guest soft-context identifiers (GSCIDs), and a memory-resident command queue (CQ) [7]. We call the implementation's translation cache the I/O translation lookaside buffer (IOTLB). A miss can invoke its page-table walker (PTW). For `IOTINVAL.VMA`, AV, PSCV, and GV indicate valid address, PSCID, and guest-context selectors, respectively. With second-stage translation disabled, `AV/PSCV/GV=0/0/0` invalidates all host address spaces; GV=0 ignores GSCID. Consequently, assigning different host PSCIDs does not isolate broad invalidation, and assigning different GSCIDs is not a host-domain solution. PSCID-scoped `0/1/0` selects one host address space, while `1/1/0` additionally selects the IOVA; both exclude global page-table entries. Our mappings have the global bit clear and second-stage translation set to Bare.

2.2 Linux DMA mappings and completion

The audited local DMA-IOMMU path maps a region through `iommu_map`, invokes a map synchronization callback, and synchronizes strict unmap; successful maps therefore also generate

invalidations. For its ordinary host path with Address Translation Services (ATS) disabled, each map or unmap region issues one invalidation and two IOFENCE commands. The audited mainline source sets PSCID and supports address invalidation. Its range-command path has priority when supported; otherwise the single-address path falls back to domain scope when its computed count is zero or exceeds 512, with further non-leaf capability conditions [5]. This is not a threshold for invalidating all devices. Mainline also supports deferred flushing through its generic page-table domain integration, subject to strict-mode configuration and allocation success. These statements describe the frozen source revision, not a kernel executed in our simulation. For an ordinary empty-slot 4 KiB map, that mainline path does not flush; strict unmap uses an address-and-PSCID invalidation followed by one PR=PW=1 IOFENCE. RISC-V's policy choice defaults to strict, but this does not establish that DMA-IOMMU is compiled in: if it is absent, the core defaults to an identity domain.

IOFENCE completion must be interpreted against the implementation. The original handler advances CQ head when it fetches a command, before decoding it, and does not implement the PR/PW read/write-drain requests. The driver model uses an invalidation followed by two AV=0 fences, polling the head after each submission; fetching the second fence in this serial, ATS-disabled handler establishes completion of the preceding invalidation/fence sequence. Unmap follows confirmed DMA completion, with PR=PW=0; this is a limited completion protocol, not a full IOFENCE conformance result.

3 METHODOLOGY

3.1 Platform and experimental boundaries

The parent platform integrates CVA6, four iDMA engines, and a RISC-V IOMMU; the reported invalidation test instantiates two real iDMA engines and the full IOMMU command/translation path. Figure 1 separates this exercised path from the parent platform's CPU/Linux context. The CPU and Linux do not run: a test driver schedules software operations, updates page tables in model memory, and performs real AXI register accesses. The baseline uses an unpartitioned 16-entry tree-PLRU IOTLB without hit bypass. Its frozen IOMMU revision includes fixes for write routing/progress, page-table leaf handling, counters, and AXI boundary handling; the common write-priority fix can affect service order. The CQ handler and IOTLB logic are byte-identical to the selected upstream implementation. This is an adapted and corrected baseline, not a claim that an unmodified complete upstream SoC was evaluated.

The behavioral DDR model has one FIFO read-return queue without ID-based reordering; the head waits 16 cycles, then returns an 8-byte beat per cycle. For an empty queue with the receiver ready, address acceptance to the first read response is 17 cycles, not 16. All invalidation modes use this same deterministic model. Cycles below are simulation clock steps, without conversion to seconds or FPGA timing.

3.2 Workload and comparisons

Initiator A performs 64 aligned 4 KiB COPY operations; neighbor B performs 256, retaining a fixed source and either three or seven

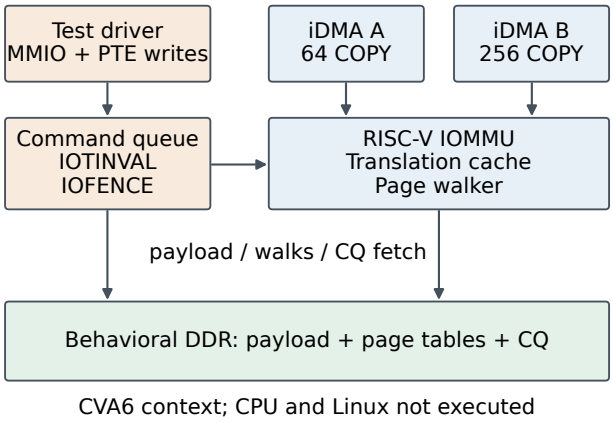


Figure 1: Exercised request path. Software operations come from the test driver; DMA, IOMMU translation, and CQ command fetches use RTL. The workload counts are read from the existing result CSV. The CPU/Linux context is not executed.

destination pages, for four- or eight-page footprints. Each A copy maps and unmaps its source and destination regions. A uses fresh descending IOVAs and does not recycle physical pages during the test. B first executes 64 warm-up copies, followed by residency checks and destination coverage checks with zero additional walks; this warm-up is outside the common measurement start. The main window retains A's cold translations and all measured command/DMA activity. Same-domain cases share the first-stage root and PSCID; different-domain cases use distinct roots and PSCIDs.

M0 persistently maps all A pages before the window and issues no invalidations inside it. It retains A's streaming replacement pressure, so it is a no-invalidation comparison rather than a reproduction of map/unmap software costs. M1 uses broad invalidation on every map and unmap. M2 scopes each operation to A's PSCID, and M3 scopes it to both A's PSCID and page address. M4 retains broad map-side invalidation but batches unmaps every $N = 16$ or 64 unmapped page regions, draining the remainder at the end and withholding reuse until synchronization. Those batch sizes model bounded delayed unmap, not Linux's actual default timer/ring policy, and do not imply that the local development driver supports deferred flushing. The original matrix contains 24 configurations and 4 controls that suppress map-side flushing; each recorded configuration was executed once.

The separate mainline-derived study assumes enabled DMA-IOMMU, translated domains, one CPU's flush queue, and pre-existing page-table levels with leaf-only unmaps. Because the original handler omits PR/PW, its test-only overlay holds fence completion until a snapshot of completion-interface accepted AR/AW requests reaches modeled DDR RLAST/write-response completion, used as a conservative global ordering point. That completion model is disclosed rather than treated as upstream RTL support or universal hardware timing. The snapshot can omit an already translated request waiting before that interface accepts

it; no such omission is established for the recorded runs, but the overlay is not a complete PR/PW safety implementation.

3.3 Attribution and time accounting

At each real CQ invalidation, the observer saves resident keys, computes the selected scope, and checks after the edge that the corresponding valid bits cleared. The first subsequent miss to such a key is invalidation-attributed; previously unseen keys are cold, and remaining misses without an invalidation mark are classified as replacement misses. This is an operational classification of misses, not an additive causal decomposition of completion time. Lookup episodes and accepted walk starts are counted separately. Raw event and per-copy records check complete data coverage, read/write beats, write responses, and hardware completion; a final DONE flag alone is insufficient.

We report T_A from the common start through A's last unmap/fence completion, and T_B through B's last hardware COPY-DONE. The joint makespan is $\max(T_A, T_B)$; adding the two device times is a different measure. The test includes MMIO polling, CQ fetches, DMA, and model-memory cycles, but excludes CPU execution time for page-table updates. Frozen source manifests, per-run identities, summaries, and compressed raw traces are retained with the existing results.

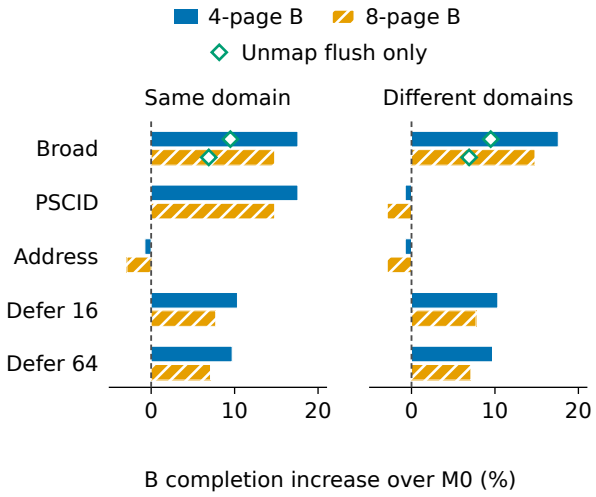


Figure 2: Neighbor B completion normalized to matching M0. Domain relation and footprint are kept explicit; M2 protects B only across distinct PSCIDs. M4 batches unmaps while retaining map-side broad invalidation.

4 RESULTS

4.1 Broad invalidation reaches the neighbor

Figure 2 normalizes B's complete time to M0 with the same footprint and domain relation. M0 takes 199740 and 204875 cycles for the four- and eight-page footprints, with 0 and 65 walks, respectively. M1 increases completion time to 234869 and 235189 cycles, or 17.6% and 14.8%. Its 322 and 326 walks are all invalidation-attributed misses; the M0 walks are not. The different-domain and same-domain measurements are identical for broad invalidation, consistent with its all-host scope. Domain separation therefore provides no invalidation isolation for M1 in this host first-stage configuration.

M2 leaves the same-domain neighbor as exposed as M1, but across domains it eliminates invalidation-attributed misses; M3 does so for both domain relations. The protected cases have 0 and 8 total B walks and complete in 198241 and 198930 cycles. The eight-page residual walks are classified as replacement, so zero invalidation misses does not mean zero misses overall. The lower eight-page walk count than M0 reflects a changed replacement history, not an additional category of negative invalidation misses; the dead-entry control below tests a narrower explanation.

4.2 Protecting B changes A's completion cost

Figure 3 includes A's final synchronization. M1 takes 194749 cycles for A, whereas the protective M2/M3 cases take 213675 or 214261 cycles, increasing T_A by 9.7%–10.0%. A's walk count stays at 128, and the fine-scope and broad per-operation modes issue the same numbers of invalidation and fence commands. Thus fewer neighbor misses do not, by themselves, predict a shorter initiator completion time. Follow-up diagnosis retains the original PR=PW=0, two-fence protocol: when A runs alone, broad and fine invalidations take the same 62 cycles from submission through first-fence completion in every recorded round. With the eight-page neighbor, the protective mode adds 19512 A cycles; its flush-busy occupancy rises by 37854, including 38503 more cycles in CQ FETCH/REGISTER, while PTW, other-copy, and host/configuration occupancy fall. The complete mode difference is therefore a co-run interaction, also conditioned on B's warm prefix, rather than extra invalidation-decode cycles; these occupancy differences are not individually recoverable causal costs. In particular, this diagnosis does not measure PR/PW drain, which was disabled in these runs.

4.3 Delayed unmap only partly mitigates broad flushing

Table 1 reports measured command counts, walks, and A time. M4 reduces invalidation/fence counts from 256/512 to 136/272 at $N = 16$, or 130/260 at $N = 64$. B still exceeds M0 complete time by 7.7%–10.3% and 7.1%–9.7%, respectively. Batching unmap therefore only partly addresses the broad-flush interference in this test. In the separate unmap-only-flush controls, B has 193/197 walks and 218700/219020 cycles, compared with M1's 322/326 walks and 234869/235189 cycles. Omitting map-side flushing materially changes the measurement; these controls isolate a component and do not replace the full map/unmap path.

4.4 Mainline-derived policies and dead translations

The mainline-derived strict policy gives B 204205 and 209940 cycles, 2.235% and 2.472% above M0; same- and different-domain values are identical. A completes in 164845 and 144795 cycles, including its final unmap synchronization. Figure 4 distinguishes these results from the development-policy matrix: map flushing, fence

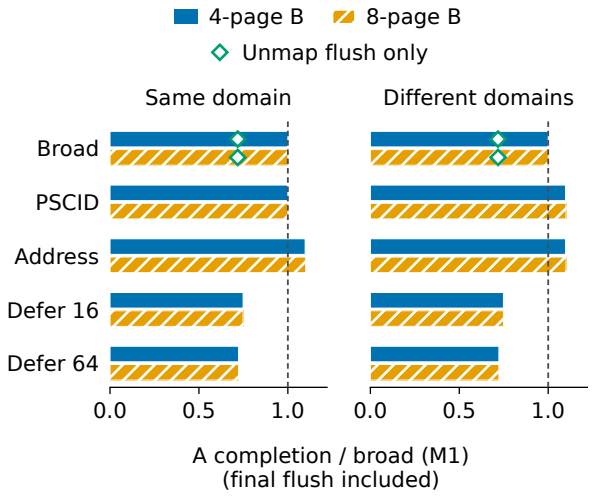


Figure 3: Initiator A completion including final unmap/fence, with M1 as the reference. These original policies use PR=PW=0 and two fences per flush; they are distinct from the mainline-derived model.

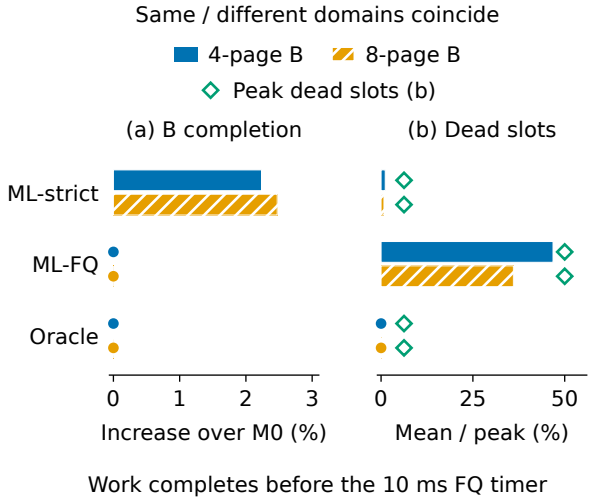


Figure 4: Mainline-derived model under explicit DMA-IOMMU assumptions and a test-only PR/PW completion overlay. FQ and ORACLE match M0 during a window that ends before the first timer flush; this is not a steady-state FQ result.

Table 1: Command and completion accounting from the existing CSV. A time includes final synchronization. M2 here uses distinct domains; same-domain M2 equals M1. M3 values hold for both domain relations.

Mode	IOTINVAL / fences	B walks (4 / 8 pages)	T_A (cycles) (4 / 8 pages)
M0	0 / 0	0 / 65	81,739 / 86,639
M1	256 / 512	322 / 326	194,749 / 194,749
M2, different	256 / 512	0 / 8	213,675 / 214,261
M3	256 / 512	0 / 8	213,675 / 214,261
M4, $N = 16$	136 / 272	210 / 214	145,718 / 145,718
M4, $N = 64$	130 / 260	198 / 202	140,756 / 140,756

count, and PR/PW semantics all differ, so the comparison is not an isolated change of scope bits.

ML-FQ uses a deferred flush queue (FQ) for unmap synchronization; ORACLE adds immediate, wait-free invalidation of dead A entries to otherwise identical FQ behavior. Both match M0's B times, 199740 and 204875, so ORACLE improves B completion by 0% over FQ in all four domain/footprint cases. FQ's mean dead occupancy is nevertheless 46.819%/36.028% of all sixteen slots. The tested PLRU victim selection does not consult valid bits, and clearing never-revisited A entries changes neither subsequent active hits nor replacement choices; the paired service traces are cycle-identical. Thus this particular dead-entry population is not evidence of recoverable performance loss, and the result does not motivate a dead-entry hint mechanism.

The deferred queue reaches only 128 entries against 255 usable entries, and both devices finish before its first 10 ms timer deadline. The timer is modeled as 500000 cycles under a declared 50 MHz/HZ=1000 equivalence; subsequent flushing completes at cycle 501475, reported separately from A's earlier API completion. This short workload does not measure interference from periodic FQ flushing or a full queue. The reverse effect of a global PR/PW fence making the unmapping device wait for a neighbor remains unmeasured; the legacy diagnosis does not establish it.

4.5 A separate trap in partitioned tree-PLRU

The replacement ablation uses a different fixture: P assigns two entries to the neighbor and fourteen to A; PN additionally enables an early hit bypass. Both retain the fixture's common pre-commit data-structure-first arbitration rule. Each device performs 64 copies, and A cycles through R destination pages with a fixed source; complete time includes cold initialization, configuration, refill, completion discovery, and final quiescence. These measurements cannot be merged numerically with the unpartitioned invalidation baseline.

At $R = 8$, A's nine-page footprint causes 37 walks and the two devices together cause 39 under the original masked tree (V0). Direct victim tracing shows A's hot source staying in way 8, all 30 valid evictions targeting ways 2 and 3, and seven eligible entries remaining unused. The extension masks eligible ways without rebalancing the underlying tree; the result demonstrates that a fourteen-entry quota does not guarantee nine-page residency for this trace. This is our platform's partition extension: the examined upstream implementation has no such way mask and does not have this particular masked-tree problem.

V1 approximately rebalances the eligible leaves while preserving the prior touch/fill semantics; V2 uses exact least-recently-used replacement, invalid-entry priority, and most-recently-used insertion. At $R = 8$, total walks fall from 39 to 25 with V1 and to 11 with V2. V2 reduces complete time by 12.44% for P and 22.37% for PN; at $R = 16$, all policies still perform 67 walks and complete time is

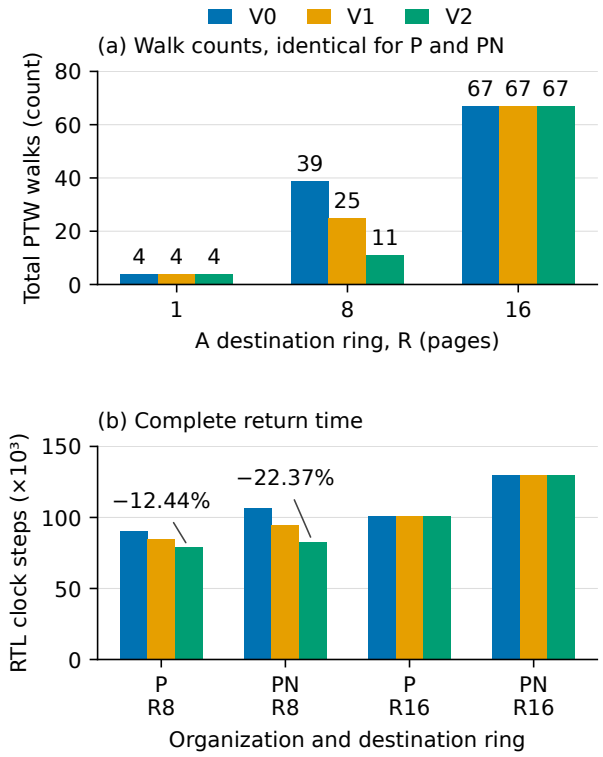


Figure 5: Separate partition-extension ablation. V0 is masked tree-PLRU, V1 approximately rebalances eligible leaves, and V2 changes replacement/insertion semantics. Complete time is the fixture's T_{return}, including final quiet; it differs from the main experiment's device times.

unchanged. Figure 5 retains these non-improving cases; the intervention bundles several replacement choices and does not isolate invalid-first insertion alone.

5 DISCUSSION

5.1 Implications for drivers and IOMMU design

The tested hardware already supports scoped invalidation, and audited mainline Linux already uses it. For a driver designer, these results support auditing actual scope bits, map callbacks, and fallback paths before treating device domains as performance isolation. For an IOMMU designer, they support measuring both devices' complete times alongside miss counts and command traffic. The observed A penalty makes a miss-only objective incomplete; distinguishing CQ-fetch interference from PR/PW completion effects remains necessary before generalizing the diagnosis. For partition design, eligible-entry counts should be accompanied by observed occupancy and victim distributions, and replacement alternatives should be compared with complete-time accounting.

5.2 Limitations

All reported timing comes from RTL simulation with behavioral DDR, not FPGA or physical-memory timing; CPU and Linux execution are absent, and software behavior is supplied by the test driver. The synthetic aligned 4 KiB COPY workloads do not establish application behavior. In particular, A's fresh descending IOVAs can differ from Linux's range-cache reuse, and the batching sizes do not reproduce its timer-driven flush queue. The invalidation matrix records one deterministic execution per configuration, so it provides no statistical uncertainty or sensitivity to different memory systems. The PLRU fixture has its own service model and research extensions; it provides a local counterexample to quota-based reasoning, not an upstream bug or a general replacement-policy ranking. Only one IOMMU implementation is characterized. The replacement, invalidation, and complete-time conclusions cannot be transferred directly to other IOMMUs. The mainline-derived baseline models selected source behavior, not a booted default kernel. Its PR/PW completion overlay does not establish a general physical ordering cost, and the global-fence neighbor-wait hypothesis remains pending.

6 RELATED WORK

Early IOTLB work examines DMA mapping costs and cache-management choices; we cite its workshop version [1]. rIOMMU uses sequential ring-buffer access to replace hierarchical translation with a circular flat table; Kurth et al. combine compiler-generated translation prefetching, parallel miss helpers, and DMA burst tracking/retry [4, 6]. MASK addresses translation and memory scheduling across concurrent GPU applications [2]. Our experiments instead quantify invalidation scope and its per-device completion tradeoff in a specific RISC-V IOMMU, and separately expose a platform-specific partition implementation pitfall. The RISC-V IOMMU extended abstract provides implementation context, without supplying a performance comparison here [8]. Intel VT-d already specifies domain- and page-selective IOTLB invalidation with scoped read/write-drain guarantees [3]; these are minimum completion guarantees, not proof that implementations never wait for other domains. We therefore make no novelty claim for adding scoped drain, and report no Intel hardware performance comparison. [TODO: 待核文献: complete verified iDMA and CVA6 bibliographic records before adding their scholarly citations.]

7 CONCLUSION

Broad host invalidation, rather than capacity pressure alone, dominates the neighbor's observed translation interference in the tested workload. Scoped invalidation removes invalidation-attributed misses and shifts measured completion cost toward the initiator; unmap batching provides partial relief while map-side flushing remains. The separate PLRU ablation shows why partition capacity must be checked against actual replacement behavior. The mainline-derived short-run result is smaller than the broad-flush result, and ideal dead-entry removal provides no B speedup; periodic FQ and global-fence effects remain outside the established findings.

Availability and declarations. Frozen sources, logs, and analysis inputs are retained locally with the reported runs. [TODO: 需要作者决定: release URL and artifact packaging; author contributions, funding, conflicts of interest, and ethics declaration.] This draft and its plots were prepared with AI assistance; the authors must review and take responsibility for the submitted version.

REFERENCES

[1] Nadav Amit, Muli Ben-Yehuda, and Ben-Ami Yassour. 2010. IOMMU: Strategies for Mitigating the IOTLB Bottleneck. In *Sixth Annual Workshop on the Interaction between Operating Systems and Computer Architecture (WIOSCA)*. https://nadav.amit.zone/publications/pdfs/amit2010iommu.pdf Author manuscript, HAL inria-00493752, version 1.

[2] Rachata Ausavarungnirun, Vance Miller, Joshua Landgraf, Saugata Ghose, Jayneel Gandhi, Adwait Jog, Christopher J. Rossbach, and Onur Mutlu. 2018. MASK: Redesigning the GPU Memory Hierarchy to Support Multi-Application Concurrency. In *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '18)*. ACM, 503–518. https://doi.org/10.1145/3173162.3173169

[3] Intel Corporation 2024. *Intel Virtualization Technology for Directed I/O Architecture Specification* (revision 5.0 ed.). Intel Corporation. https://cdrdv2-public.intel.com/831418/vt-directed-io-spec.pdf Order Number D51397-017, Section 6.5.4.

[4] Andreas Kurth, Pirmin Vogel, Andrea Marongiu, and Luca Benini. 2018. Scalable and Efficient Virtual Memory Sharing in Heterogeneous SoCs with TLB Prefetching and MMU-Aware DMA Engine. In *2018 IEEE 36th International Conference on Computer Design (ICCD)*. IEEE, 292–300. https://doi.org/10.1109/ICCD.2018.00052

[5] Linux source n.d.. Linux RISC-V IOMMU Driver. Source code, torvalds/linux. https://github.com/torvalds/linux/blob/58161ead29e50d70a317ea26b472e0c7508e69da/drivers/iommu/riscv/iommu.c Commit 58161ead29e50d70a317ea26b472e0c7508e69da; snapshot audited 17 October 2026.

[6] Moshe Malka, Nadav Amit, Muli Ben-Yehuda, and Dan Tsafrir. 2015. rIOMMU: Efficient IOMMU for I/O Devices that Employ Ring Buffers. In *Proceedings of the Twentieth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '15)*. ACM, 355–368. https://doi.org/10.1145/2694344.2694355

[7] RISC-V International 2026. *RISC-V IOMMU Architecture Specification* (version 1.0.1 ed.). RISC-V International. https://docs.riscv.org/reference/iommu/v20260222/_attachments/riscv-iommu.pdf Ratified, 22 February 2026.

[8] Manuel Rodríguez, Francisco Costa, Bruno Sá, and Sandro Pinto. 2023. Open-source RISC-V Input/Output Memory Management Unit (IOMMU) IP. RISC-V Summit Europe, Barcelona, Spain. https://riscv-europe.org/summit/2023/media/proceedings/posters/2023-06-07-Manuel-RODR%C3%8DGUEZ-abstract.pdf Two-page extended abstract.